System Analysis

Problem Identification and Current System:

Currently, there are two methods of procurement: online and offline. From the context of Bangladesh, none of these is proper. Offline procurement has flaws like corruption and discrimination. Online portals like EGP(government platform), bdtender.com, and alltender.com don't have a solid approach to the problem. The lackluster UI and even worse user experience have made this side of business unbearable.

Proposed System:

We propose ProcureNext, which addresses the requirements and expectations identified and also integrates modern technology to make the process of procurement more seamless and hassle-free.

Business Process:

ProcureNext is going to employ a point-based business scheme. When uploading a tender, the buyers will spend points. On the vendor side, they can view the tender documents. When submitting any bid, they need to spend a certain number of points to do that. This will ensure proper accountability on both the buyer and vendor sides. Also proper interface will be provided to update the point to the taka(BDT) scheme, packages for points, and a safe payment process when buying points.

System Design:

ProcureNext employs a microservices architecture to ensure high availability, fault tolerance, and independent scalability. Each domain operates independently of the others. This is particularly important for AI/NLP modules, which will require high computational power (CPU resources) and must scale independently. Also, to provide a seamless user experience, we need to use message queuing.

Core System Components:

- **Presentation Layer (Frontend):** A responsive UI delivering role-based dashboards for Admins, Buyers, Vendors, and Public Users, along with a public portal for tender browsing
- **API Gateway and Core Microservices:** Centralized routing that handles authentication before directing traffic to isolated backend services. (User Management, Tender Management, Bidding, Payments, etc.)
- **ML Layer:** This will be an independent service dedicated to heavy intelligent functionality such as semantic search, vendor matching, recommendation system, and document parsing.
- **Asynchronous Task Queue:** The system offloads heavy computational tasks, such as bulk pdf upload(and then sending to ML Layer), to a background task queue using Celery and Redis. Redis acts as the message broker, preventing HTTPS timeout, i.e. main web thread from blocking.

Technology Stack

Component	Selected Technology	Justification
Frontend	Next.js, React, TailwindCSS	Enables fast development, strong component ecosystems, and highly responsive user interfaces.
Backend Framework	FastAPI	Python serves as the native language for AI/ML. FastAPI will be used for seamless integration with the ML microservice.
Database	PostgreSQL and pgvector	Unifies relational data and vector embeddings in a single database, eliminating the cost and complexity of maintaining a specialized vector database while ensuring strict ACID compliance.
AI / NLP Models	all-MiniLM-L6-v2, PyMuPDF	The lightweight sentence transformer runs efficiently on local CPU/RAM environments to maintain privacy. PyMuPDF offers superior speed and layout analysis for parsing complex tender PDFs
Cache & Queue	Redis + Celery	Essential for offloading heavy, long-running ML tasks (like PDF parsing and embedding generation) to background queues, preventing the main web threads from freezing.
Object Storage	MinIO / S3	Provides durable, scalable, and cost-effective cold storage for large files such as raw tender PDFs, NDAs, and audit reports, keeping the primary database lean.
Deployment	Docker	Ensures the application and its system-level ML

		dependencies run identically across development and production environments.

Data Storage and Processing Policy

- **Hybrid Storage Strategy:** The system splits data horizontally. Relational metadata, user profiles, and transaction logs are stored in PostgreSQL alongside semantic vector embeddings via the pgvector extension. Bulky unstructured artifacts are routed entirely to the Object Storage bucket.
- **Asynchronous Processing Flow:** When a user uploads a tender document, the file is immediately passed to the background task queue (Celery/Redis). This allows the ML service to run OCR, extract text, and generate vector embeddings asynchronously, returning a "Success" state to the frontend via polling without disrupting the user experience.

Security, Compliance, and Payments

- **Access Control:** The system enforces strict Role-Based Access Control (RBAC) middleware. Additional Row-Level Security ensures users can only access their specific localized data, such as private draft bids.
- **Immutable Auditability:** To support forensic review and compliance, all critical actions (tender creation, bid submission, awards) are tracked in an append-only audit log containing the user ID, timestamp, data snapshots, and cryptographic hash signatures.
- **Monetization Integration:** Credit point purchases and refunds are processed securely via integration with the SSLCommerz payment gateway using server-side callback webhooks.

Functional Requirements (FR)

1.1 Unregistered User (Public Access)

FR-01 — General Information Access (MUST)
The system shall allow unregistered users to read-only view the platform's background, policies, legal notices, news, events, and help/support pages.
Acceptance: All public info pages are accessible without login; links are visible on the homepage.

Public users shall also see the company's "about us", contact information, and platform explanations describing how the procurement system works, and advantages.

FR-02 — Browse Active Public E-Tenders (MUST)
The system shall allow unregistered users to browse currently active public tenders. Only basic tender information shall be visible to unregistered users, including: - Buyer/company name - Tender title - Category/type - Publication and deadline dates - Basic non-descriptive summary

Detailed tender descriptions, financial terms, eligibility documents, and downloadable files shall NOT be accessible without registration.
Acceptance: Public users can view limited summaries but cannot access detailed tender documentation.

FR-03 — Advanced Search (Semantic) (MUST)
The system shall provide an advanced search for active public tenders with semantic matching and filters (category, location, deadline, budget, organization, tender visibility). Acceptance: Relevant results returned, and filters operate correctly.

FR-04 — User Registration Entry Point (MUST)

The system shall support two types of registered users within an organization:

1. **Master Account Holder (Organization Admin)** — the primary account owner representing the organization.

2. **Normal User (Organization Member)** — users who work for the organization and operate under the master account holder's authority.

A single organization can register as a Buyer, Seller, or both simultaneously under a single organization account.

Registration and Verification Requirements:

For Master Account Holder (Organization Admin):

- Email verification
- Mobile OTP verification
- Submission of personal NID for identity verification
- Submission of organizational documents:
    - Trade License
    - TIN Certificate
    - VAT Certificate
    - Any additional required regulatory documents (if applicable)

The master account holder shall be responsible for managing additional users within the organization and assigning role-based permissions (e.g., manager, evaluator, viewer, bidder).

For Normal Users (Organization Members):

- Email verification
- Mobile OTP verification
- Submission of personal NID for identity verification

Normal users shall only gain system access after being verified by the master account holder for an organization and granted the appropriate roles.

The system admin will have the responsibility of manually verifying the submitted NIDs of users, and will mark it as verified. The corresponding user will receive a notification of verification.

Acceptance:
Registration completes only after the required verification steps are passed, organizational documents are submitted by the master account holder, and user identity verification is completed for all users.

1.2 Registered Users — Common

FR-05 — Authentication & Account Management (MUST)
Secure login, logout, password reset via registered email, optional 2FA, and OTP verification for sensitive actions. Acceptance: Login, password reset, and OTP flows function correctly.

FR-06 — Profile Management & Verification (MUST)

Each company shall have: - One master account holder (Admin) - Admin ability to add users to the organization by sending them requests, and assigning various roles having different levels of privileges and access.

Affiliation Request Process

1. It is the responsibility of the administrator to send affiliation requests to the appropriate users.

2. Users have the option to either accept or decline a request sent by an administrator of an organization.

3. Each master account of an organization shall have a unique code associated with it. Users may enter this code within the system to request affiliation with the respective organization. The administrator of that organization will then review and approve or reject the request accordingly.

Acceptance: Verification status is visible.

FR-07 — Notifications & Messaging (MUST)

The system will have a chatting / messaging system, allowing registered users to communicate based on the following protocols:

Inter-Company Messaging: This service will be available tender-wise, where representatives of two companies will be able to communicate with one another, in presence of their respective company owner/admin/master account holder. It is in essence, a group chat of at least 4 people (2 from each side), and the admins can add/remove assigned users to/from this chat group. There will be no scope of one-to-one messaging between users of different companies.

Intra-Company Messaging: Users affiliated with the same company can chat with each other in a one-to-one method. Also, there will be a scope of forming groups where the admin can broadcast messages to many users at once, much like available group chat methods.

The system will have a notification system through emails for alerting about the following events:

User-specific events: Notifications related to incoming messages, acceptance of validity of submitted documents for identification.

Organization-specific events: Notifications for events like: user has requested to be affiliated with the organization, the organization has been verified by the system admin.

Organization workflow related events: Notifications for events like: acceptance of bid by an organization, being invited/enlisted, deadlines, NOA issuance, acceptance, bid-bond refunds.

Acceptance: Notifications generated for relevant events; message history persists.

1.3 Registered Users — Buyer/Procuring Entity

FR-08 — Tender Creation & Management (MUST)
Buyers can create, edit, cancel, schedule, and publish tenders. Tender properties include: - Title - Budget - Deadline - Categories - Visibility (Public / Exclusive to enlisted vendors) - Required documents list defined by buyer - Bid-bond amount (specified by buyer)

Tenders may be: - Single-item tenders, or - Packaged tenders containing multiple items/lots (e.g., cement and rod together).

Acceptance: Tender lifecycle operates correctly.

FR-09 — Vendor Matching & Recommendations (MUST)
The system shall recommend vendors based on profile matching, certifications, past performance, and relevance. Acceptance: Ranked recommendations with explanations displayed.

FR-10 — Bid Comparison & Evaluation Tools (MUST)
Buyers can compare bids side-by-side and evaluate pricing and submitted documents. Acceptance: Comparison view displays all required information.

FR-11 — Notice of Award (NOA) & Selection Workflow (MUST)
After evaluation: - Buyer selects a seller and issues a Notice of Award (NOA). - Seller must accept NOA online. - Acceptance consumes the required platform credits of the seller. - Buyer receives notification upon acceptance. Unselected sellers shall automatically receive a refund of their bid-bond.

Acceptance: NOA issuance and acceptance recorded with an audit trail.

FR-12 — Restricted Invitations (SHOULD)
Buyers may publish exclusive tenders visible only to enlisted vendors. Acceptance: Non-invited vendors cannot view exclusive tenders.

FR-13 — Contract Draft & Completion (MUST)
The system shall generate a draft contract template. Buyer and seller may edit terms and conditions.

Contract signing occurs offline(physically on legal papers). After both parties confirm contract signing within the system, platform involvement ends for that procurement cycle.

Acceptance: Contract confirmation recorded.

FR-14 — Seller Performance Reviews (SHOULD)
Buyers may submit performance reviews for sellers after contract completion. The system shall store reviews for future visibility. Acceptance: Reviews saved and linked to seller profiles.

1.4 Registered Users — Vendor-Specific

FR-15 — Tender Feed & Opportunity Alerts (MUST)
Vendors shall receive personalized dashboards showing relevant tenders and alerts. Acceptance:
Relevant tenders appear based on the profile.

FR-16 — Bid Submission & Bid-Bond Workflow (MUST)
Vendors may submit bids by: - Uploading technical and financial proposals - Providing required documents defined by buyer - Paying the specified bid-bond amount

Acceptance: Buyer can reject the bids from sellers who have not uploaded the required documents that the buyer had asked to provide, after manual verification.

FR-17 — Bid Status & History (MUST) Vendors can track bid lifecycle (submitted, under evaluation, awarded, rejected). Acceptance: Status changes timestamped.

1.5 Financial Handling
FR-18 — Credit Points & Payments (Monetization) (MUST)
Users can purchase and manage platform credit points used for bidding activities and premium features. Acceptance: Payment flow updates balances and transaction history.

FR-19 — Bid-Bond Transaction Handling (MUST)
The platform shall handle financial transactions related to bid bonds. Bid-bond funds shall be

stored within the buyer's designated account or holding mechanism (implementation model to be finalized). Refunds shall be automatically processed for unsuccessful bidders. Acceptance: Transactions recorded and refundable when applicable.

FR-20 — Tender Publishing Cost Configuration (MUST)
The system shall maintain global default pricing for publishing tenders. Administrators may apply discounts or cost adjustments for specific buyers or sellers.
Acceptance: Configurable pricing reflected during publishing.

1.6 Admin / System

FR-21 — Admin Dashboard & Moderation (MUST)
Admins can: - Manage users - Verify documents manually - Moderate disputes - Block/unblock accounts - Configure publishing costs

Acceptance: Admin actions logged.

FR-22 — Audit Trail & Immutable Logging (MUST)
All critical actions must be logged with timestamps and user identity. Acceptance: Logs retrievable and tamper-evident.

FR-23 — Dispute & Complaint Management (NOT SURE) Users may raise disputes; admins track resolution workflow. Acceptance: Dispute lifecycle visible.

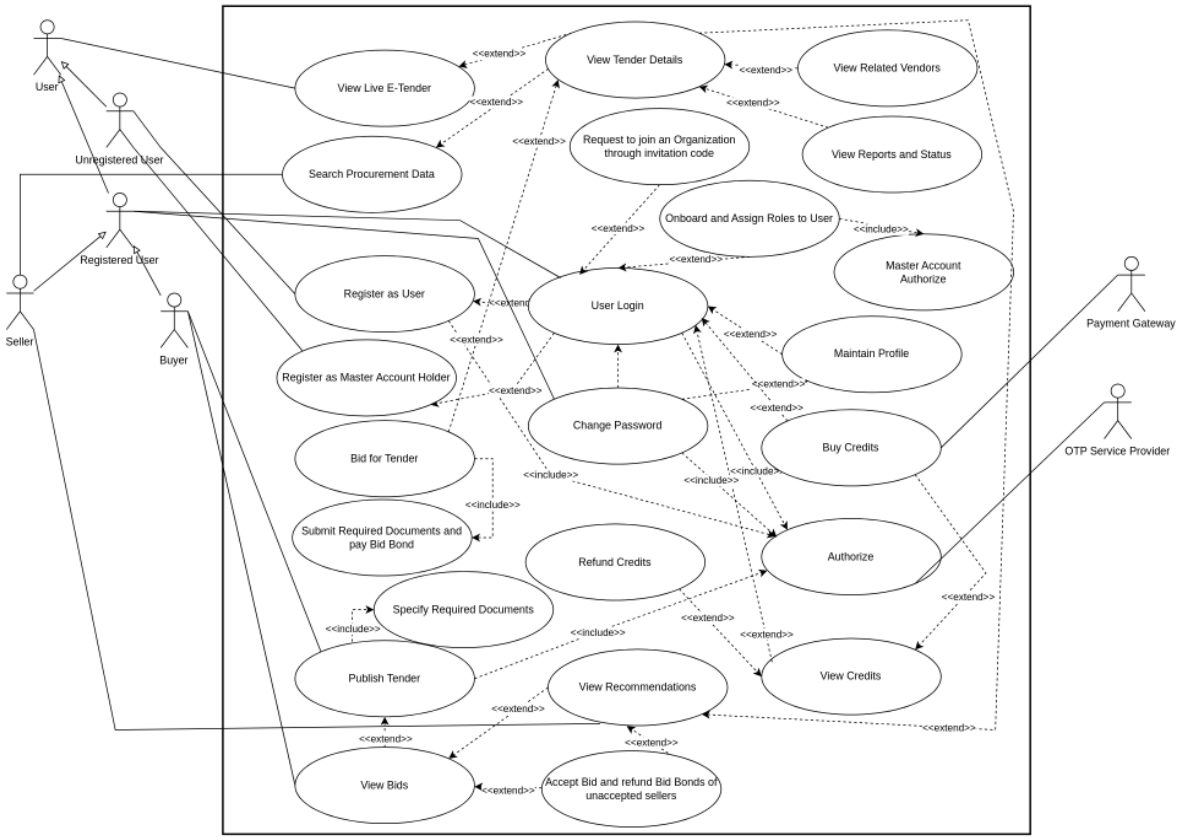
FR-24 — Reporting & Analytics (NOT SURE) Reports include tender volumes, vendor performance, system usage, and revenue analytics. Acceptance: Reports exportable.

1.7 External Integrations & APIs

FR-25 — Payment Gateway Integration (MUST)
Integration with at least one secure payment gateway. Acceptance: Successful sandbox and production transactions.

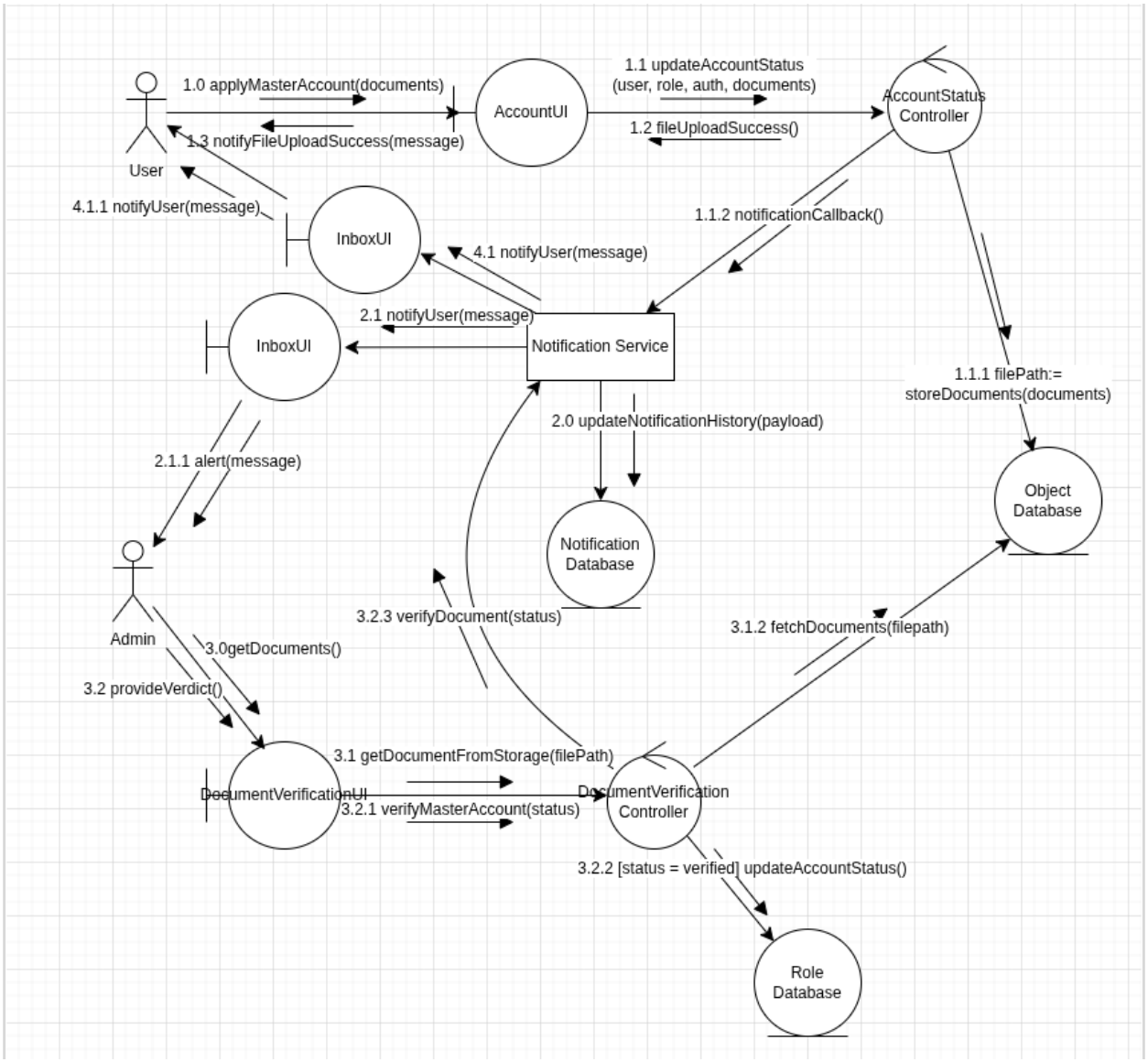
FR-26 — External Identity & Verification APIs (NOT SURE)
Optional integration with third-party identity and financial verification services. Acceptance: Verification results stored.

Formal Use Case Diagram

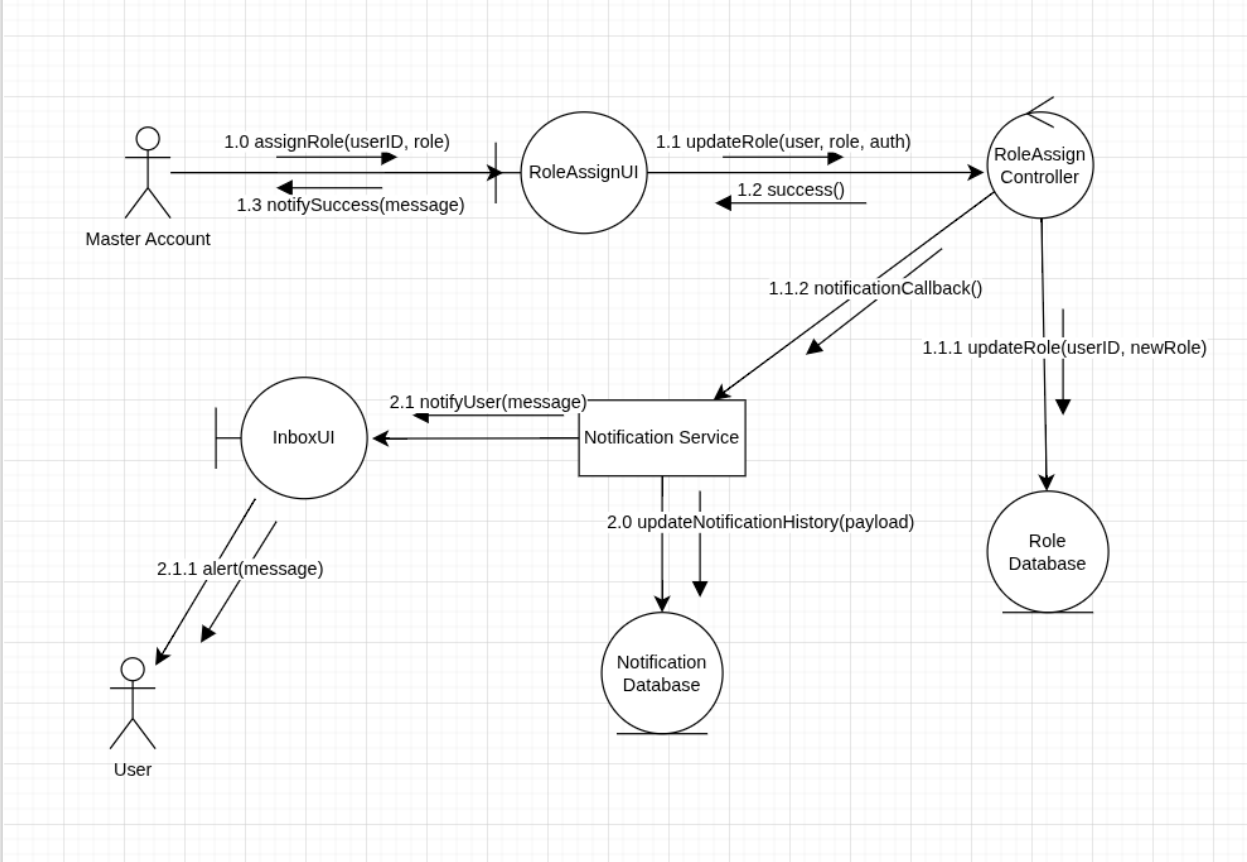


Collaboration Diagram

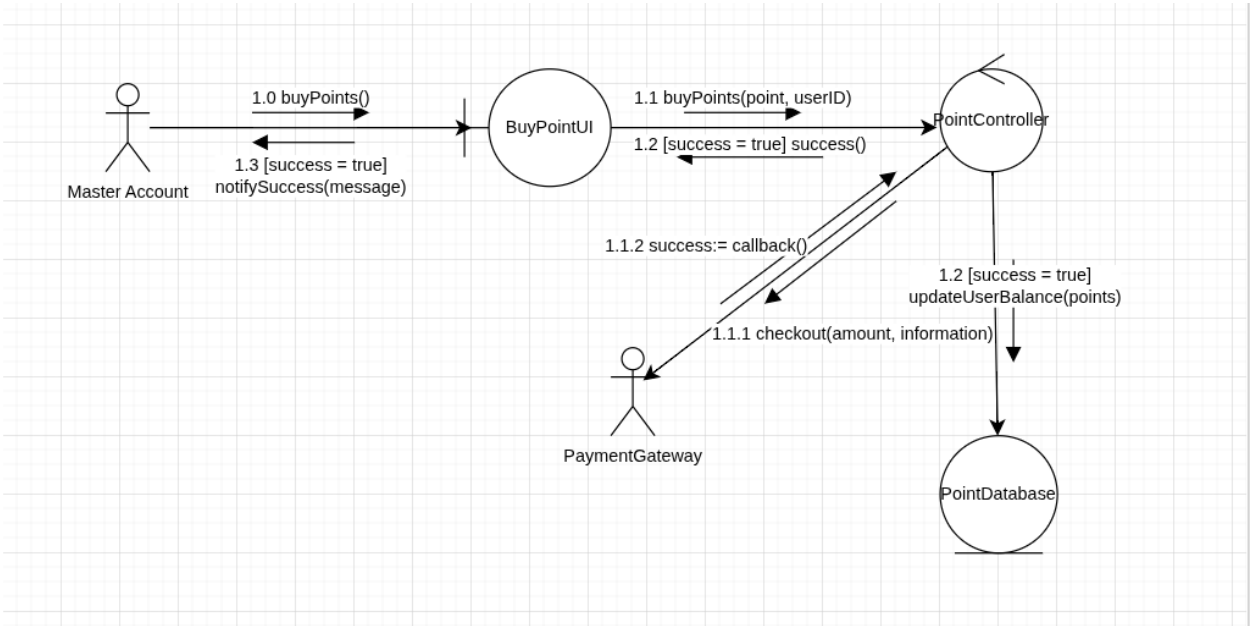
1. Master Account Registration



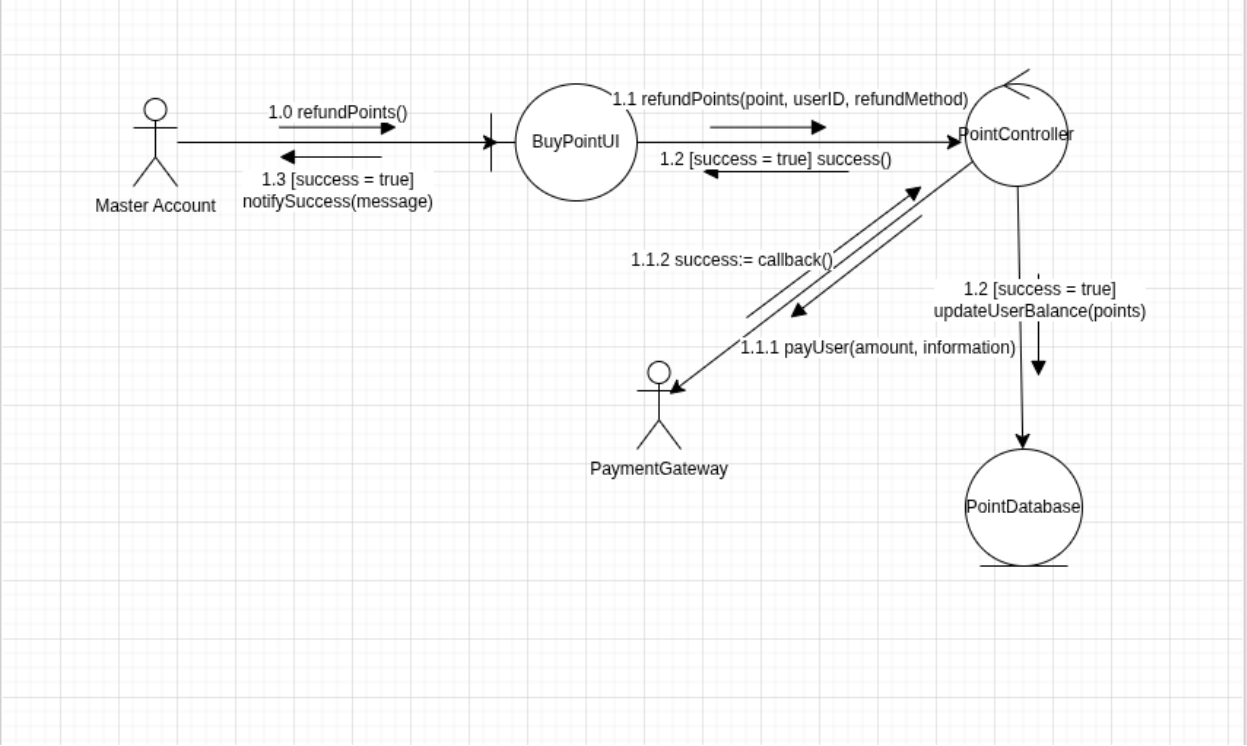
2. Role Assignment:



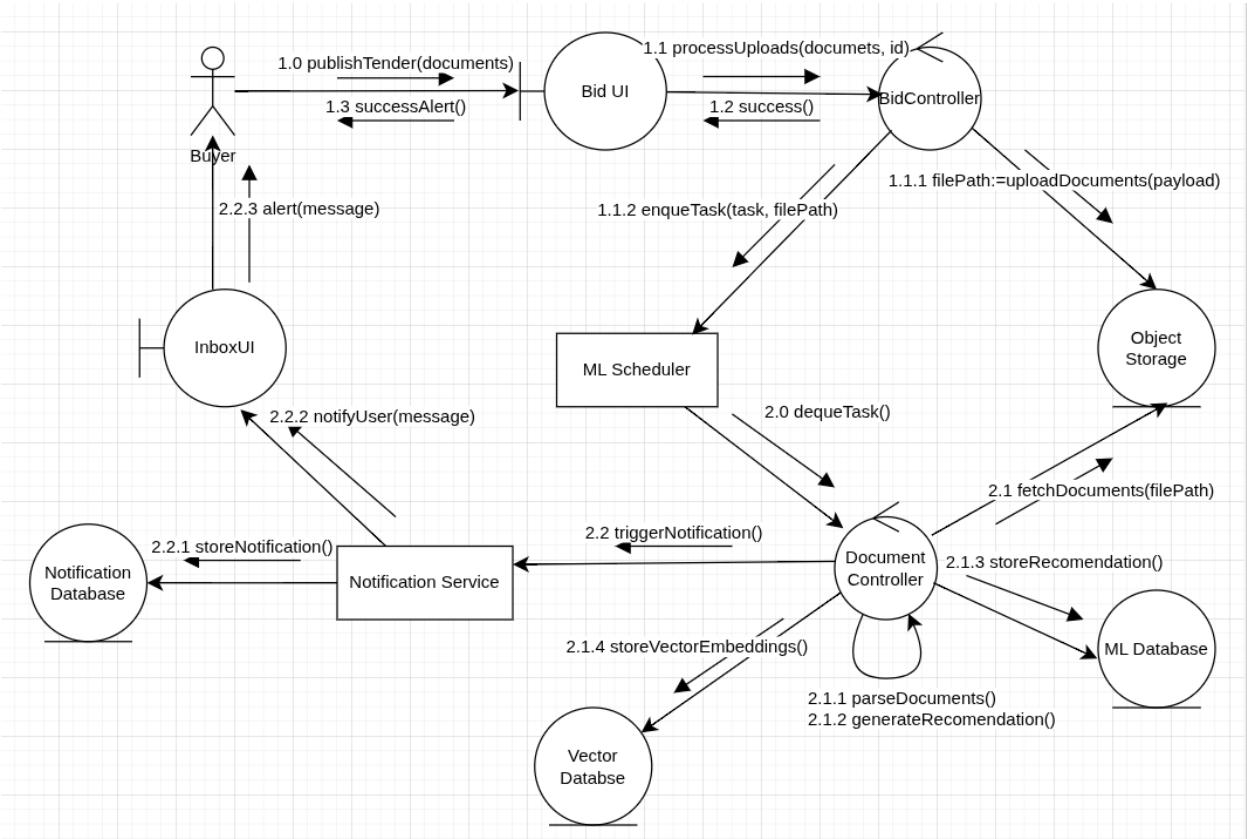
3. Buy Credits:



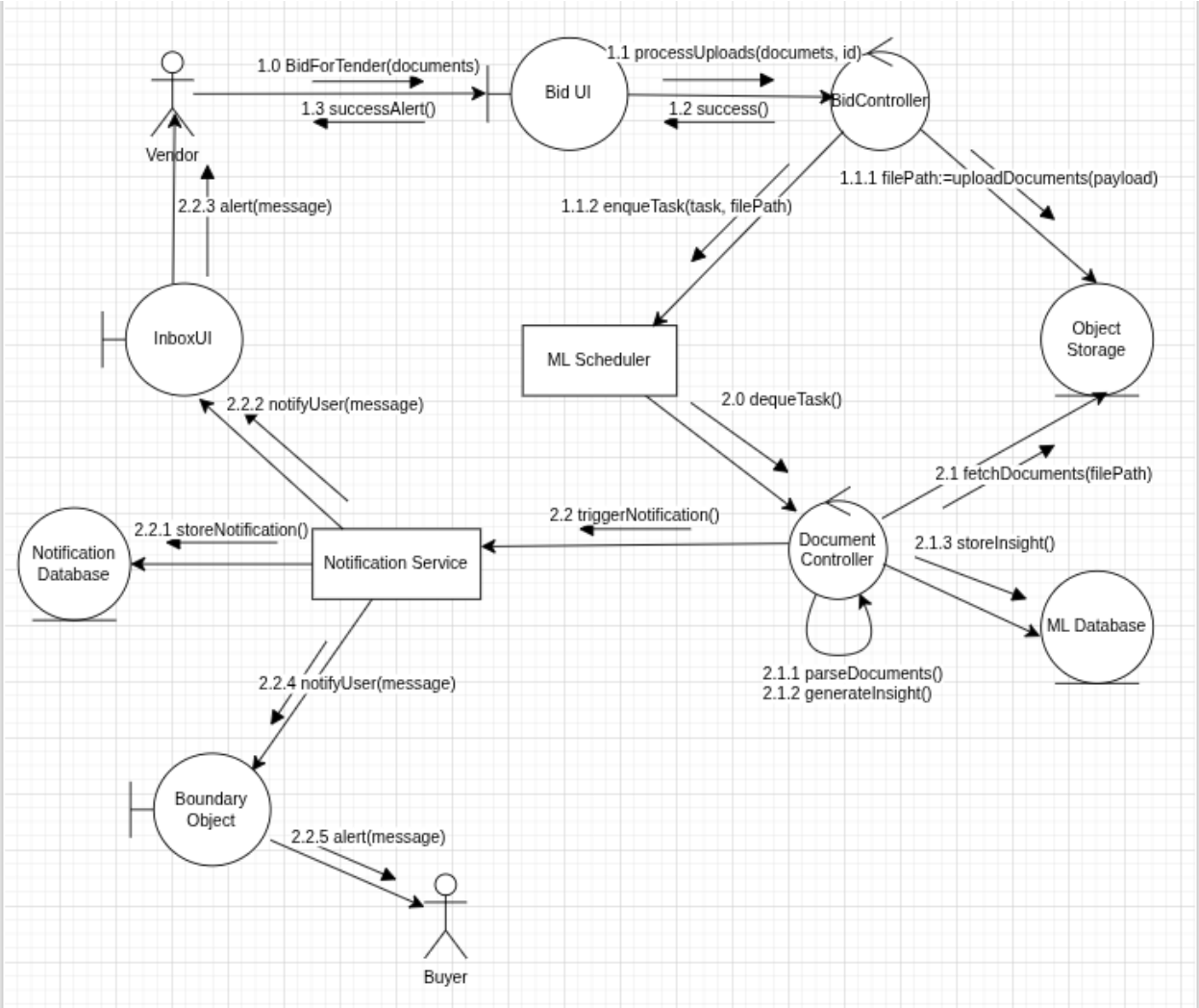
4. Refund Credits:



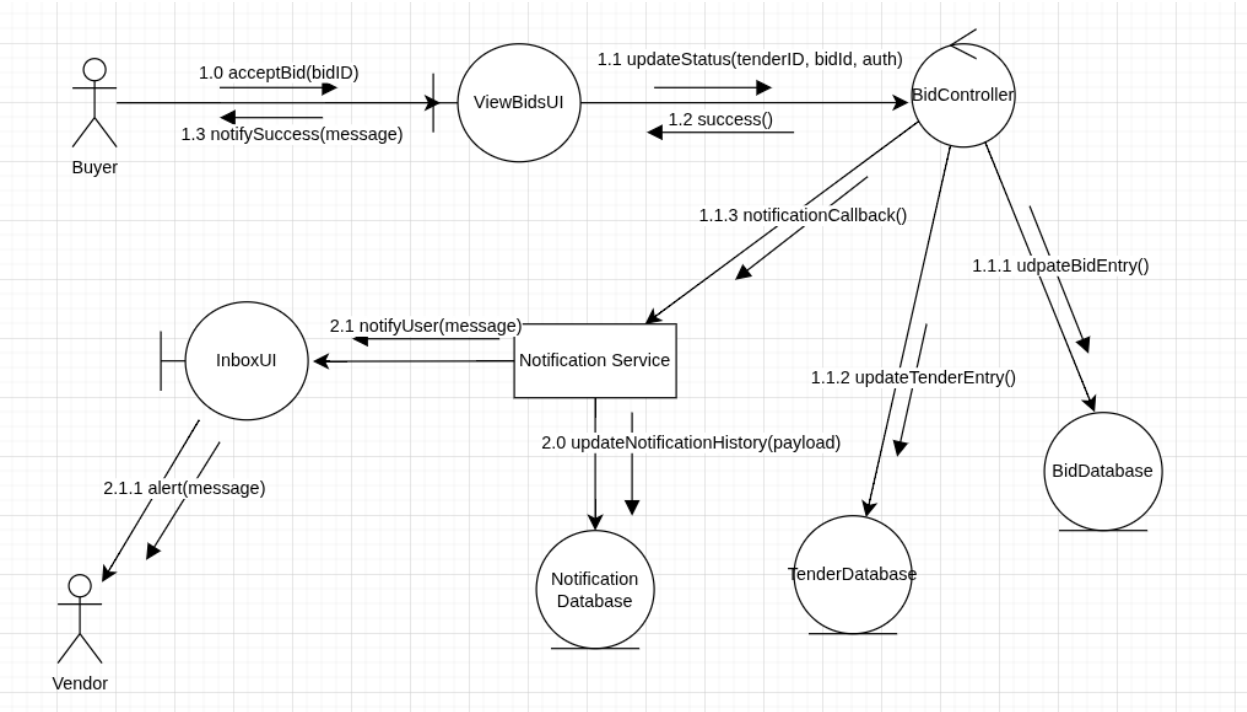
5. Publish Public Tender:



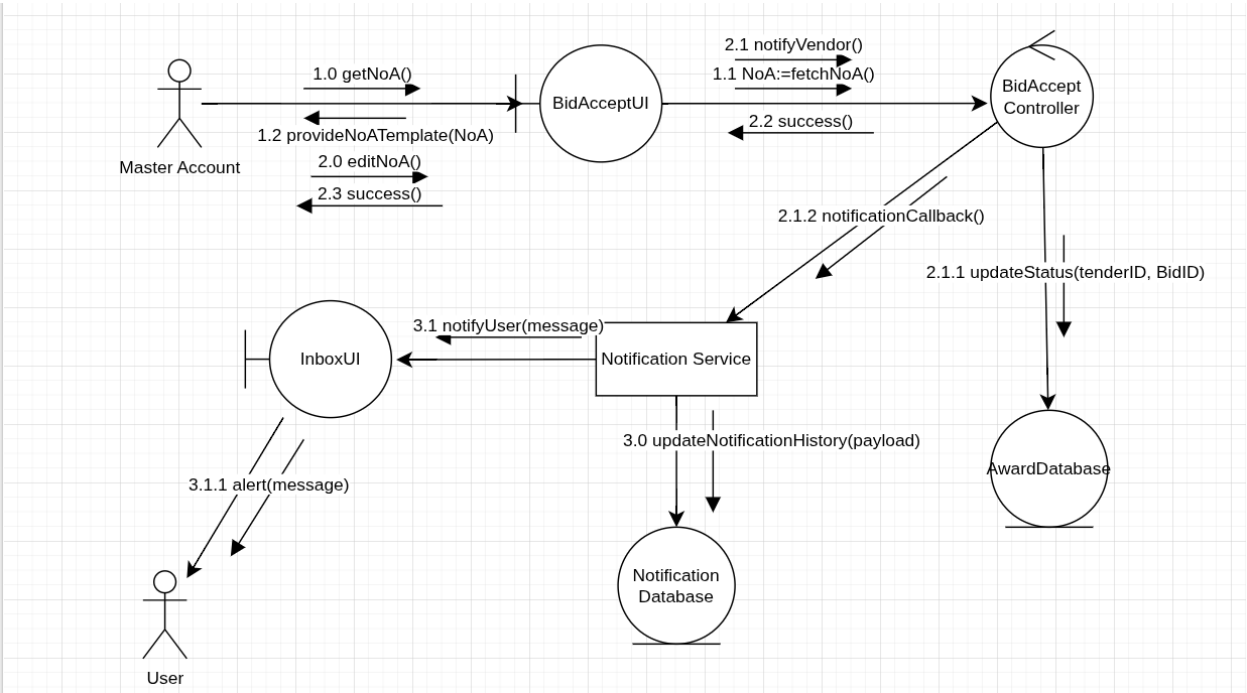
6. Bid for Tender:



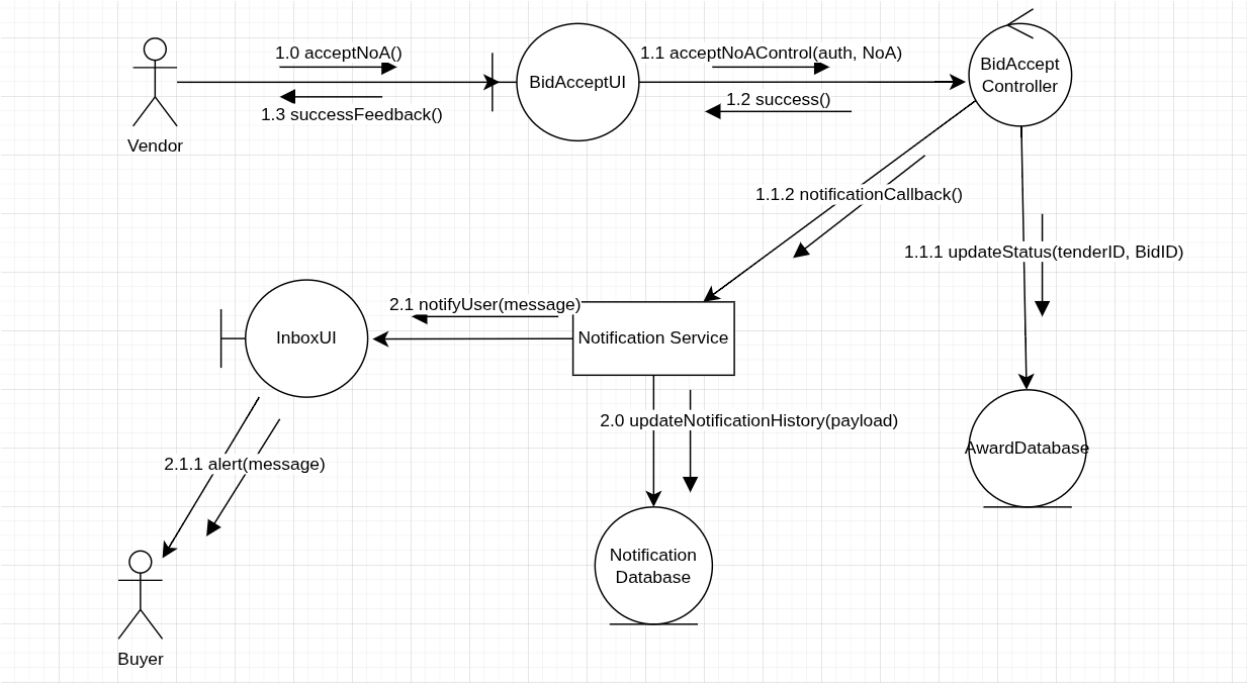
7. Accept Bid:



8. Send NoA:



9. Accept NoA:


Vendor
1.0 acceptNoA()
1.3 successFeedback()
BidAcceptUI
1.1 acceptNoAControl(auth, NoA)
1.2 success()
BidAccept Controller
1.1.2 notificationCallback()
1.1.1 updateStatus(tenderID, BidID)
2.1 notifyUser(message)
InboxUI
Notification Service
2.0 updateNotificationHistory(payload)
AwardDatabase
2.1.1 alert(message)
Notification Database
Buyer

Description of specific software modules with their IO

1. User & Identity Management Module

Handles user onboarding, authentication, role assignment, and organization affiliations.

- **Inputs:**
  - User credentials (email, password, OTP).
  - Personal Identity Documents (NID).
  - Organization Documents (Trade License, TIN, VAT).
  - Affiliation codes and role assignment commands (from Master Admin).
- **Outputs:**
  - Authentication tokens / Session cookies.
  - Verification status markers (Verified/Unverified).
  - User role permissions (Admin, Manager, Evaluator, Viewer, Bidder).
  - Approved/Rejected affiliation links between users and organizations.

2. Tender Management Module (Buyer)

Manages the complete lifecycle of a tender from creation to publication.

- **Inputs:**
  - Tender details (Title, budget, deadline, category, item/lot packages).
  - Visibility settings (Public vs. Exclusive/Restricted).
  - Required eligibility documents list and bid-bond amount.
- **Outputs:**
  - Published public tender listings.
  - Exclusive tender invitations to enlisted vendors.
  - Tender status updates (Draft, Active, Cancelled, Completed).

3. Bidding & Evaluation Module

Facilitates vendor bid submissions and buyer evaluation workflows.

- **Inputs:**
  - Vendor technical and financial proposals.
  - Uploaded vendor compliance documents.
  - Buyer evaluation scores/decisions.
  - Notice of Award (NOA) issuance commands.
- **Outputs:**
  - Timestamped bid status history (Submitted, Under Evaluation, Awarded, Rejected).
  - Side-by-side bid comparison views for buyers.
  - Notice of Award (NOA) documents to selected sellers.

4. Search & Recommendation Engine

Empowers tender discovery for public users, personalized feeds for vendors, and matching

tenders for buyers.

- **Inputs:**
  - Search queries and semantic keywords.
  - Filters (Category, location, deadline, budget, organization).
  - User profiles, certifications, and past performance history.
- **Outputs:**
  - Filtered, semantic search results.
  - Ranked vendor recommendations with matching explanations (for buyers).
  - Personalized tender feeds and opportunity alerts (for vendors).
  - Basic summary views (for unregistered public users).

5. Financial & Transaction Module

Manages the platform's internal economy, bidding costs, and bid bonds.

- **Inputs:**
  - Payment requests via external gateways.
  - NOA acceptance triggers.
  - Bid-bond deposit funds.
  - Admin pricing configurations and discounts.
- **Outputs:**
  - Updated user Credit Point balances.
  - Bid-bond transaction records (held funds).
  - Automated refund triggers (for unsuccessful bidders).

6. Contract & Performance Module

Handles post-award operations and vendor rating.

- **Inputs:**
  - Custom edits to draft terms and conditions of the contract.
  - Manual confirmations of physical/offline contract signing.
  - Seller’s performance reviews and ratings.
- **Outputs:**
  - Draft contract templates.
  - Finalized procurement cycle status.
  - Buyer reviews seller profiles.

7. Notification & Messaging Module

A cross-cutting module handling all platform communications.

- **Inputs:** System-generated events (NOA issued, bid rejected, affiliation requested, deadline approaching).
- **Outputs:** In-app notifications, Emails, and SMS alerts.

8. System Admin, Moderation & Logging Module

Provides oversight, security logging, and analytics.

- **Inputs:**
  - Admin moderation commands (block/unblock, verify documents manually).
  - User-submitted disputes/complaints.
  - Every critical action performed across the platform (for logging).
- **Outputs:**
  - Immutable, tamper-evident audit logs with timestamps.
  - Exportable analytics reports (revenue, volumes, system usage).
  - Dispute resolution status tracking.

Interaction Between Different Technology Layers

The ProcureNext system will operate on a standard N-Tier architecture. Here is how the different technological layers interact to fulfill the requirements:

1. Presentation Layer (Client / UI Interface)

- **Technologies:** Web application and potentially Mobile interfaces.
- **Interactions:**
  - Provides specialized views based on the user's state:
    - **Public Users:** Queries the API Gateway for read-only CMS pages and basic tender searches.
    - **Buyers & Vendors:** Submits complex forms (tender creation, bid document uploads) and visualizes data (dashboards, bid comparisons).
  - Intercepts actions requiring multi-factor authentication and prompts the user for OTP inputs before passing them down.

2. API Gateway & Security Layer

- **Technologies:** REST/GraphQL API Gateway, WAF (Web Application Firewall), Load Balancers.
- **Interactions:**
  - Acts as the single entry point for the Presentation Layer.
  - Validates session tokens and enforces Role-Based Access Control (RBAC). For
  - example, it ensures a Normal User cannot access Master Admin features, or that an uninvited vendor cannot access a restricted tender.
  - Routes validated requests to the appropriate Business Logic Services.

3. Business Logic Layer (Microservices / Application Services)

- **Technologies:** Backend runtimes
- **Interactions:**
  - **Workflow Orchestration:** When a vendor bids, the Bidding Service triggers the

Financial Service to deduct Credit Points, triggers the Notification Service to alert the buyer.
    - **Search & Matching:** The Tender Service communicates with a specialized Search/Semantic engine to run advanced queries and generate vendor recommendations based on profile metadata.
    - **Automated Events:** Runs scheduled background jobs (e.g., when a tender deadline passes, it triggers the Bidding module to lock submissions and initiates automated refunds for rejected bid-bonds via the Financial Module).

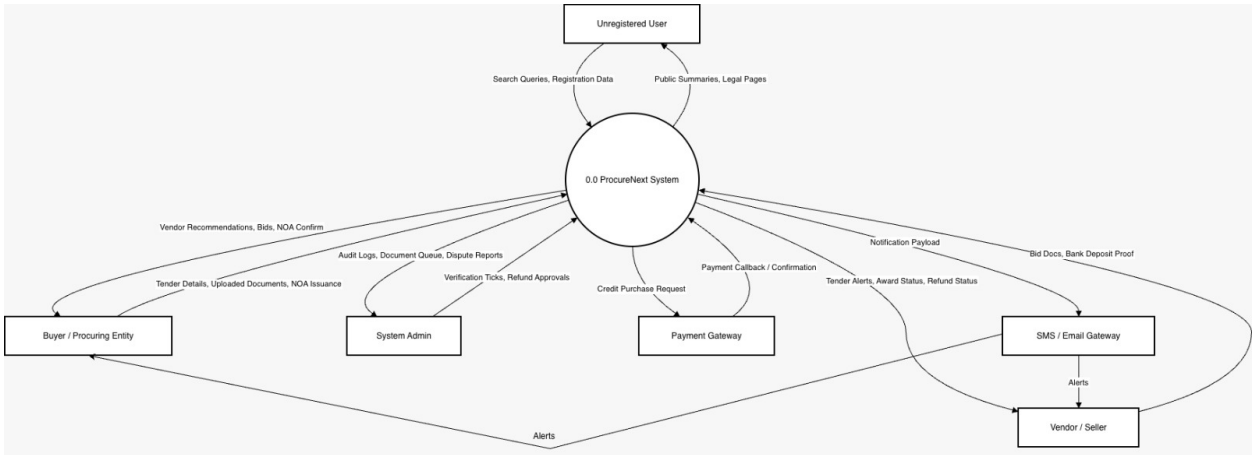
4. Data Access & Persistence Layer

- **Technologies:** Relational DB (SQL for transactions), Cloud Blob Storage (for files), and Immutable Ledger/Log DB.
- **Interactions:**
    - **File Storage:** Receives and securely stores verification documents (NID, TIN) and bid uploads. Provides secure, time-limited download URLs to authorized buyers.
    - **Immutable Logging:** The Business Logic layer asynchronously sends all critical transaction data (logins, bid submissions, NOA awards) to an append-only, tamper-evident datastore to satisfy audit requirements.

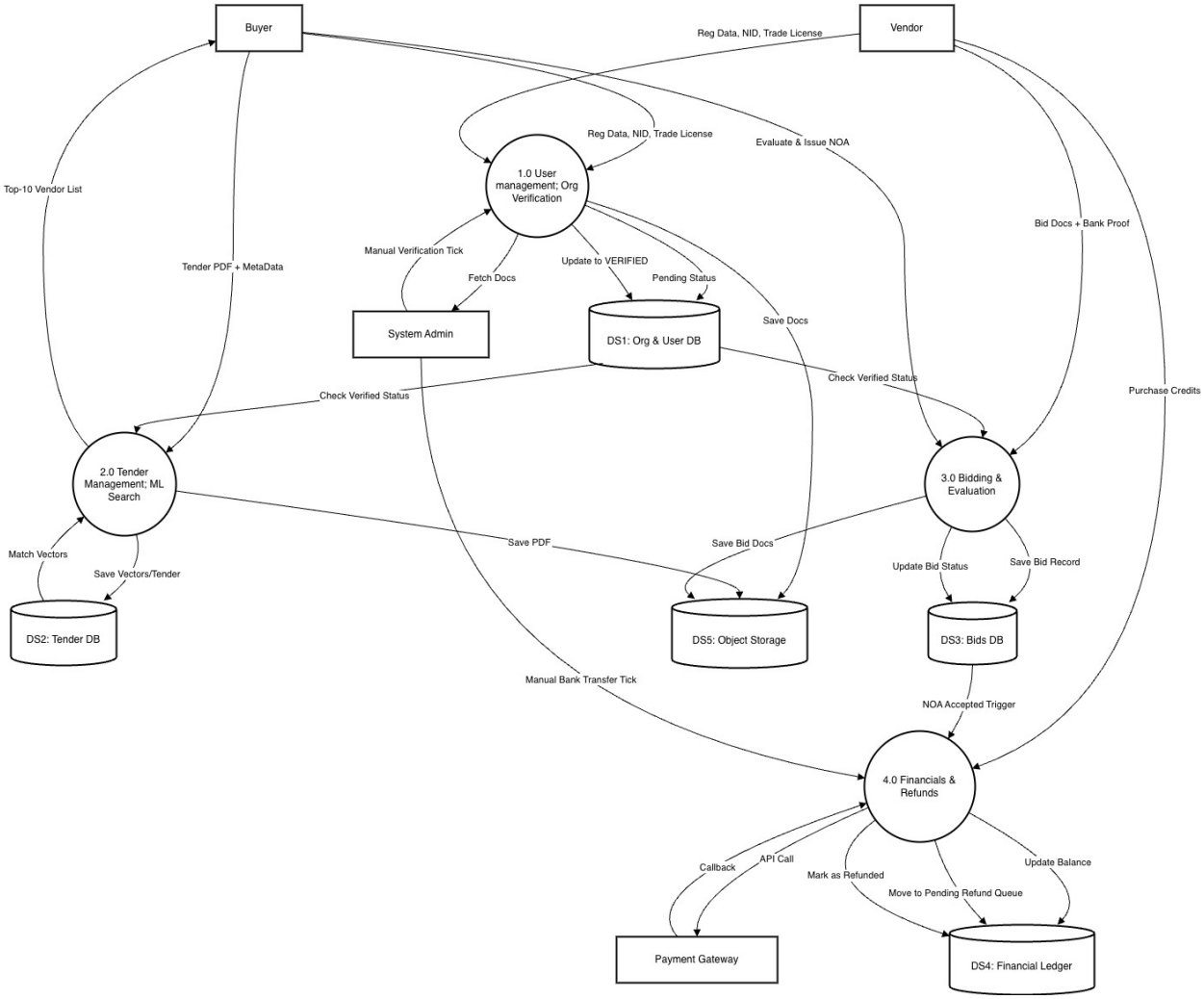
5. External Integration Layer

- **Technologies:** API Clients, Webhooks.
- **Interactions:**
    - **Payment Gateway:** The Financial Service communicates securely with external payment processors to process credit purchases and bid-bond deposits/refunds.
    - **Identity APIs:** During registration, the Identity Module interacts with third-party government/financial APIs to automatically verify NID and TIN credentials.
    - **Communication Gateway:** Sends payloads to external SMS and Email providers to deliver out-of-app notifications.

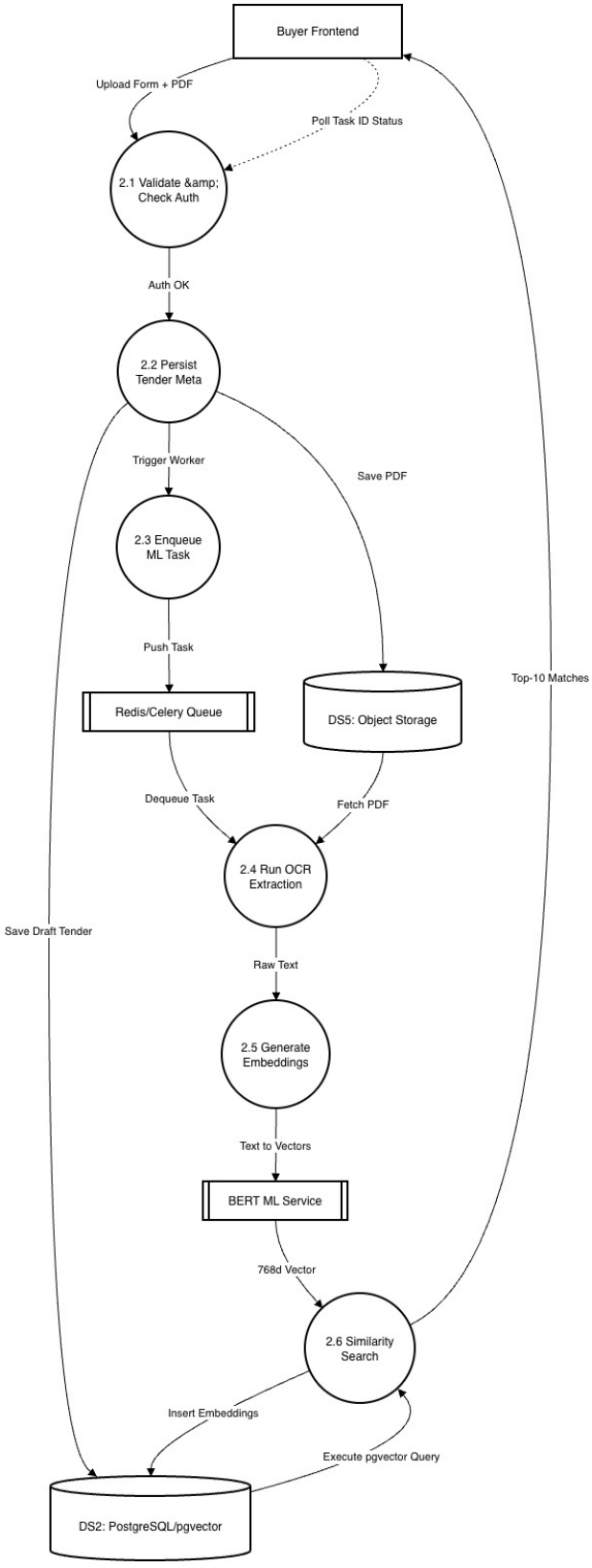
Data Flow Diagram



Level- 0


Buyer
Vendor
Reg Data, NID, Trade License
Reg Data, NID, Trade License
Evaluate & Issue NOA
1.0 User management; Org Verification
Top-10 Vendor List
Tender PDF + MetaData
Bid Docs + Bank Proof
Manual Verification Tick
Fetch Docs
Update to VERIFIED
Pending Status
System Admin
DS1: Org & User DB
Save Docs
Check Verified Status
Check Verified Status
Purchase Credits
2.0 Tender Management; ML Search
3.0 Bidding & Evaluation
Match Vectors
Save Vectors/Tender
Save PDF
Save Bid Docs
Update Bid Status
Save Bid Record
DS2: Tender DB
DS5: Object Storage
DS3: Bids DB
Manual Bank Transfer Tick
NOA Accepted Trigger
4.0 Financials & Refunds
Callback
API Call
Mark as Refunded
Move to Pending Refund Queue
Update Balance
Payment Gateway
DS4: Financial Ledger

Level - 1


Buyer Frontend
Upload Form + PDF
Poll Task ID Status
2.1 Validate & Check Auth
Auth OK
2.2 Persist Tender Meta
Trigger Worker
Save PDF
2.3 Enqueue ML Task
Push Task
Redis/Celery Queue
DS5: Object Storage
Top-10 Matches
Dequeue Task
Fetch PDF
2.4 Run OCR Extraction
Save Draft Tender
Raw Text
2.5 Generate Embeddings
Text to Vectors
BERT ML Service
768d Vector
2.6 Similarity Search
Insert Embeddings
Execute pgvector Query
DS2: PostgreSQL/pgvector

Level - 2

Schema

```
-- ============================================================
-- eProcurement Platform — PostgreSQL Database Schema
-- ============================================================

-- Enable required extensions
CREATE EXTENSION IF NOT EXISTS pgvector;

-- ============================================================
-- ENUMS
-- ============================================================

CREATE TYPE user_status          AS ENUM ('Active', 'Suspended', 'Pending');
CREATE TYPE admin_role_type      AS ENUM ('SuperAdmin', 'PlatformAdmin');
CREATE TYPE organization_type    AS ENUM ('Buyer', 'Vendor');
CREATE TYPE verification_status  AS ENUM ('Pending', 'Verified', 'Rejected');
CREATE TYPE role_in_org          AS ENUM ('Owner', 'ProcurementOfficer', 'Finance', 'Viewer');

CREATE TYPE review_status_enum   AS ENUM ('Pending', 'Approved', 'Rejected');
CREATE TYPE tender_visibility    AS ENUM ('Public', 'Restricted');
CREATE TYPE tender_status        AS ENUM ('Draft', 'Published', 'Closed', 'Awarded', 'Cancelled');
CREATE TYPE invitation_status    AS ENUM ('Pending', 'Accepted', 'Declined');
CREATE TYPE bid_status           AS ENUM ('Draft', 'Submitted', 'UnderEvaluation', 'Accepted',
'Rejected', 'Withdrawn');
CREATE TYPE security_type        AS ENUM ('BankGuarantee', 'Escrow', 'WalletHold');
CREATE TYPE contract_status      AS ENUM ('Active', 'Completed', 'Terminated');
CREATE TYPE completion_status    AS ENUM ('OnTime', 'Delayed', 'Incomplete', 'Disputed');
CREATE TYPE transaction_type     AS ENUM ('Purchase', 'Deduct', 'Refund');
CREATE TYPE notification_ref_type AS ENUM ('TENDER', 'BID', 'CONTRACT', 'SYSTEM');
```

```
CREATE TYPE chat_room_status      AS ENUM ('Active', 'Closed', 'Archived');
CREATE TYPE chat_role             AS ENUM ('Admin', 'Member');
CREATE TYPE suggestion_status     AS ENUM ('Pending', 'Invited', 'Dismissed');
CREATE TYPE procurement_nature_val AS ENUM ('Goods', 'Works', 'Services', 'Consultancy');
CREATE TYPE procurement_method_val AS ENUM ('OTM', 'RFQ', 'RFP', 'ReverseAuction', 'Direct');

-- ============================================================
-- CORE USER & AUTH TABLES
-- ============================================================

CREATE TABLE users (
  user_id        SERIAL         PRIMARY KEY,
  email          VARCHAR(255)   UNIQUE NOT NULL,
  nid            NUMERIC        UNIQUE NOT NULL,
  date_of_birth  TIMESTAMP      NOT NULL,
  password_hash  TEXT           NOT NULL,
  phone          VARCHAR(30),
  is_2fa_enabled BOOLEAN        DEFAULT FALSE,
  status         user_status    DEFAULT 'Pending',
  last_login_at  TIMESTAMP,
  created_at     TIMESTAMP      DEFAULT NOW(),
  updated_at     TIMESTAMP      DEFAULT NOW()
);

CREATE TABLE admins (
  admin_id   SERIAL         PRIMARY KEY,
  user_id    INT            NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  admin_role admin_role_type NOT NULL
);
```

```
CREATE TABLE user_verification (
   user_id          INT          PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
   verified_by      INT          REFERENCES admins(admin_id),
   nid_front_file_path TEXT,
   nid_back_file_path  TEXT,
   review_status     review_status_enum DEFAULT 'Pending',
   verified_at       TIMESTAMP
);

-- ============================================================
-- ORGANIZATION TABLES
-- ============================================================

CREATE TABLE organizations (
   organization_id     SERIAL            PRIMARY KEY,
   primary_contact     INT               REFERENCES users(user_id),
   organization_name    VARCHAR(255)       NOT NULL,
   organization_type    organization_type   NOT NULL,
   address             TEXT,
   website             VARCHAR(255),
   description         TEXT,
   verification_status  verification_status DEFAULT 'Pending',
   tin_number          VARCHAR(50),
   bin_number          VARCHAR(50),
   credit_balance      INT               DEFAULT 0,
   unique_join_code     VARCHAR(50)       UNIQUE,
   org_embedding       VECTOR(768),
   created_at          TIMESTAMP         DEFAULT NOW()
);
```

```
CREATE TABLE organization_employees (
  org_user_id     SERIAL         PRIMARY KEY,
  organization_id INT            NOT NULL REFERENCES organizations(organization_id) ON DELETE
CASCADE,
  user_id         INT            NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  role_in_org     role_in_org    NOT NULL,
  joined_at       TIMESTAMP      DEFAULT NOW(),
  UNIQUE (organization_id, user_id)
);

CREATE TABLE document_types (
  type_id     SERIAL         PRIMARY KEY,
  type_name   VARCHAR(100)   UNIQUE NOT NULL,
  description TEXT,
  is_active   BOOLEAN        DEFAULT TRUE,
  created_at  TIMESTAMP      DEFAULT NOW()
);

-- Seed initial known types (extendable at any time)
INSERT INTO document_types (type_name) VALUES
  ('TradeLicense'),
  ('TIN'),
  ('VAT'),
  ('RJSC');

CREATE TABLE organization_documents (
  document_id     SERIAL            PRIMARY KEY,
  organization_id INT               NOT NULL REFERENCES organizations(organization_id) ON
DELETE CASCADE,
  reviewed_by     INT               REFERENCES admins(admin_id),
```

```
  document_type_id INT            NOT NULL REFERENCES document_types(type_id),

  file_path      TEXT             NOT NULL,
  review_status  review_status_enum  DEFAULT 'Pending',
  review_notes   TEXT,
  reviewed_at    TIMESTAMP,
  uploaded_at    TIMESTAMP        DEFAULT NOW()
);

CREATE TABLE enlisted_vendors (
  org_id         INT       NOT NULL REFERENCES organizations(organization_id) ON DELETE
CASCADE,
  enlisted_org_id INT      NOT NULL REFERENCES organizations(organization_id) ON DELETE
CASCADE,
  enlisted_by    INT       REFERENCES organization_employees(org_user_id),
  enlisted_at    TIMESTAMP  DEFAULT NOW(),
  PRIMARY KEY (org_id, enlisted_org_id)
);

-- ============================================================
-- LOOKUP / REFERENCE TABLES
-- ============================================================

CREATE TABLE procurement_nature (
  nature_id  SERIAL              PRIMARY KEY,
  name       procurement_nature_val  UNIQUE NOT NULL
);

CREATE TABLE procurement_method (
  method_id  SERIAL              PRIMARY KEY,
```

```
    method_code procurement_method_val  UNIQUE NOT NULL,
    description TEXT
);

CREATE TABLE tender_categories (
    category_id     SERIAL          PRIMARY KEY,
    parent_id       INT             REFERENCES tender_categories(category_id),
    category_name   VARCHAR(255)    NOT NULL,
    UNIQUE (parent_id, category_name)
);

-- ============================================================
-- TENDER TABLES
-- ============================================================

CREATE TABLE tenders (
    tender_id           SERIAL          PRIMARY KEY,
    buyer_id            INT             NOT NULL REFERENCES organizations(organization_id),
    created_by          INT             NOT NULL REFERENCES organization_employees(org_user_id),
    category_id         INT             REFERENCES tender_categories(category_id),
    nature_id           INT             REFERENCES procurement_nature(nature_id),
    method_id           INT             REFERENCES procurement_method(method_id),
    title               VARCHAR(500)    NOT NULL,
    description         TEXT            NOT NULL,
    visibility_type     tender_visibility DEFAULT 'Public',
    budget_min          NUMERIC(18,2),
    budget_max          NUMERIC(18,2),
    security_required   BOOLEAN         DEFAULT FALSE,
    security_valid_until DATE,
    proposal_valid_until DATE,
```

```
    publish_datetime    TIMESTAMP,
    submission_deadline TIMESTAMP,
    status              tender_status   DEFAULT 'Draft',
    embedding           VECTOR(768),
    created_at          TIMESTAMP       DEFAULT NOW(),
    updated_at          TIMESTAMP       DEFAULT NOW()
);

CREATE TABLE tender_documents (
    tender_doc_id   SERIAL      PRIMARY KEY,
    tender_id       INT         NOT NULL REFERENCES tenders(tender_id) ON DELETE CASCADE,
    file_name       VARCHAR(255),
    file_path       TEXT,
    uploaded_at     TIMESTAMP   DEFAULT NOW()
);

CREATE TABLE tender_invitations (
    invitation_id       SERIAL              PRIMARY KEY,
    tender_id           INT                 NOT NULL REFERENCES tenders(tender_id) ON DELETE
CASCADE,
    vendor_org_id       INT                 NOT NULL REFERENCES organizations(organization_id),
    invited_at          TIMESTAMP           DEFAULT NOW(),
    invitation_status   invitation_status   DEFAULT 'Pending',
    UNIQUE (tender_id, vendor_org_id)
);

CREATE TABLE tender_vendor_suggestions (
    suggestion_id   SERIAL              PRIMARY KEY,
    tender_id       INT                 NOT NULL REFERENCES tenders(tender_id) ON DELETE CASCADE,
    vendor_org_id   INT                 NOT NULL REFERENCES organizations(organization_id),
```

```
    similarity_score NUMERIC(5,4),
    suggested_at    TIMESTAMP          DEFAULT NOW(),
    status          suggestion_status  DEFAULT 'Pending'
);

-- ============================================================
-- BID TABLES
-- ============================================================

CREATE TABLE bids (
    bid_id          SERIAL     PRIMARY KEY,
    vendor_org_id   INT        NOT NULL REFERENCES organizations(organization_id),
    submitted_by    INT        NOT NULL REFERENCES organization_employees(org_user_id),
    tender_id       INT        NOT NULL REFERENCES tenders(tender_id),
    financial_amount NUMERIC(18,2),
    status          bid_status  DEFAULT 'Draft',
    submitted_at    TIMESTAMP   DEFAULT NOW(),
    updated_at      TIMESTAMP   DEFAULT NOW(),
    UNIQUE (tender_id, vendor_org_id)
);

CREATE TABLE bid_documents (
    bid_doc_id      SERIAL     PRIMARY KEY,
    bid_id          INT        NOT NULL REFERENCES bids(bid_id) ON DELETE CASCADE,
    doc_type_id     INT        NOT NULL REFERENCES document_types(type_id),
    file_path       TEXT,
    uploaded_at     TIMESTAMP   DEFAULT NOW()
);

CREATE TABLE bid_securities (
```

```
    security_id         SERIAL          PRIMARY KEY,
    bid_id              INT             NOT NULL REFERENCES bids(bid_id) ON DELETE CASCADE,
    security_amount     NUMERIC(18,2),
    security_type       security_type,
    bid_security_doc_path TEXT,
    submitted_at        TIMESTAMP DEFAULT NOW(),
    valid_until         DATE
);

-- ============================================================
-- AWARD & CONTRACT TABLES
-- ============================================================

CREATE TABLE awards (
    award_id        SERIAL      PRIMARY KEY,
    winning_bid_id  INT         NOT NULL REFERENCES bids(bid_id),
    awarded_by      INT         NOT NULL REFERENCES organization_employees(org_user_id),
    tender_id       INT         NOT NULL REFERENCES tenders(tender_id),
    remarks         TEXT,
    awarded_at      TIMESTAMP   DEFAULT NOW()
);

CREATE TABLE contracts (
    contract_id             SERIAL          PRIMARY KEY,
    award_id                INT             NOT NULL REFERENCES awards(award_id),
    contract_value          NUMERIC(18,2),
    signed_at               TIMESTAMP,
    contract_document_path  TEXT,
    estimated_end_date      DATE,
    status                  contract_status DEFAULT 'Active'
```

```
);

CREATE TABLE vendor_performance (
    performance_id      SERIAL              PRIMARY KEY,
    vendor_org_id       INT                 NOT NULL REFERENCES organizations(organization_id),
    contract_id         INT                 NOT NULL REFERENCES contracts(contract_id),
    rating              NUMERIC(2,1)        CHECK (rating BETWEEN 1 AND 5),
    feedback            TEXT,
    completion_status   completion_status,
    recorded_at         TIMESTAMP           DEFAULT NOW(),
    embedding           VECTOR(768)
);

-- ============================================================
-- PAYMENT & CREDIT TABLES
-- ============================================================

CREATE TABLE payments (
    transaction_id          SERIAL          PRIMARY KEY,
    organization_id         INT             NOT NULL REFERENCES organizations(organization_id),
    amount                  NUMERIC(18,2)   NOT NULL,
    gateway_transaction_id  VARCHAR(255),
    gateway_validation_id   VARCHAR(50),
    status                  VARCHAR(50),
    paid_at                 TIMESTAMP       DEFAULT NOW()
);

CREATE TABLE credit_transactions (
    transaction_id      SERIAL              PRIMARY KEY,
    organization_id     INT                 NOT NULL REFERENCES organizations(organization_id),
```

```
    payment_id          INT             REFERENCES payments(transaction_id),
    amount              NUMERIC(18,2)   NOT NULL,
    transaction_type    transaction_type    NOT NULL,
    payment_reference   VARCHAR(255),
    balance_after       NUMERIC(18,2),
    created_at          TIMESTAMP       DEFAULT NOW()
);

CREATE TABLE credit_discounts (
    org_id              INT         PRIMARY KEY REFERENCES organizations(organization_id),
    issued_by           INT         REFERENCES admins(admin_id),
    discounted_credit_price NUMERIC(18,2),
    valid_from          TIMESTAMP,
    valid_until         TIMESTAMP,
    is_active           BOOLEAN     DEFAULT TRUE
);

-- ============================================================
-- NOTIFICATION TABLES
-- ============================================================

CREATE TABLE notification_types (
    type_id     SERIAL          PRIMARY KEY,
    type_name   VARCHAR(100)    UNIQUE NOT NULL,
    description TEXT,
    is_active   BOOLEAN         DEFAULT TRUE,
    created_at  TIMESTAMP       DEFAULT NOW()
);
```

```
CREATE TABLE notifications (
   notification_id SERIAL              PRIMARY KEY,
   created_by      INT                 REFERENCES organization_employees(org_user_id),
   reference_type  notification_ref_type,
   reference_id    BIGINT,
   type_id         INT                 NOT NULL REFERENCES notification_types(type_id),
   title           VARCHAR(255)        NOT NULL,
   message         TEXT,
   created_at      TIMESTAMP           DEFAULT NOW()
);

CREATE TABLE notification_recipients (
   recipient_id    SERIAL      PRIMARY KEY,
   notification_id INT         NOT NULL REFERENCES notifications(notification_id) ON DELETE
CASCADE,
   org_user_id     INT         NOT NULL REFERENCES organization_employees(org_user_id),
   is_read         BOOLEAN     DEFAULT FALSE,
   read_at         TIMESTAMP
);

CREATE TABLE user_notifications (
   notification_id SERIAL      PRIMARY KEY,
   user_id         INT         NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
   title           VARCHAR(255),
   message         TEXT,
   created_at      TIMESTAMP   DEFAULT NOW()
);

-- ============================================================
-- MESSAGING TABLES
```

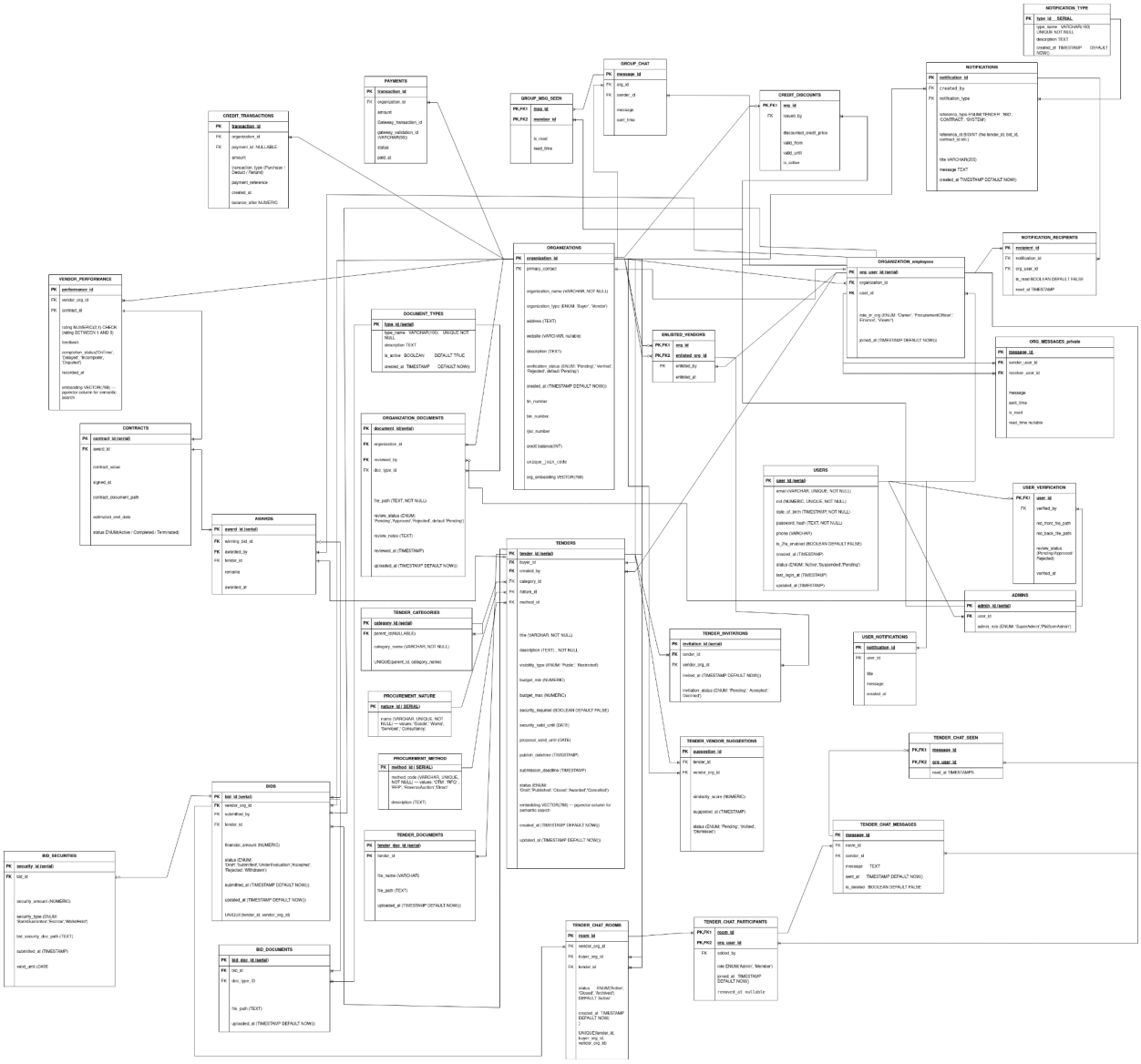
```
-- ============================================================

-- Direct (private) messages between users
CREATE TABLE org_messages_private (
  message_id     SERIAL     PRIMARY KEY,
  sender_user_id  INT       NOT NULL REFERENCES users(user_id),
  receiver_user_id INT      NOT NULL REFERENCES users(user_id),
  message        TEXT,
  sent_time      TIMESTAMP  DEFAULT NOW(),
  is_read        BOOLEAN    DEFAULT FALSE,
  read_time      TIMESTAMP
);

-- Organisation-level group chat
CREATE TABLE group_chat (
  message_id  SERIAL     PRIMARY KEY,
  org_id      INT        NOT NULL REFERENCES organizations(organization_id) ON DELETE
CASCADE,
  sender_id   INT        NOT NULL REFERENCES organization_employees(org_user_id),
  message     TEXT,
  sent_time   TIMESTAMP  DEFAULT NOW()
);

CREATE TABLE group_msg_seen (
  msg_id      INT        NOT NULL REFERENCES group_chat(message_id) ON DELETE CASCADE,
  member_id   INT        NOT NULL REFERENCES organization_employees(org_user_id),
  is_read     BOOLEAN    DEFAULT FALSE,
  read_time   TIMESTAMP,
  PRIMARY KEY (msg_id, member_id)
);
```

```
-- Tender-scoped chat rooms (buyer ↔ vendor per tender)
CREATE TABLE tender_chat_rooms (
  room_id       SERIAL        PRIMARY KEY,
  vendor_org_id  INT          NOT NULL REFERENCES organizations(organization_id),
  buyer_org_id   INT          NOT NULL REFERENCES organizations(organization_id),
  tender_id     INT           NOT NULL REFERENCES tenders(tender_id),
  status        chat_room_status DEFAULT 'Active',
  created_at    TIMESTAMP     DEFAULT NOW(),
  UNIQUE (tender_id, buyer_org_id, vendor_org_id)
);

CREATE TABLE tender_chat_participants (
  room_id    INT       NOT NULL REFERENCES tender_chat_rooms(room_id) ON DELETE
CASCADE,
  org_user_id INT      NOT NULL REFERENCES organization_employees(org_user_id),
  added_by   INT       REFERENCES organization_employees(org_user_id),
  role       chat_role  DEFAULT 'Member',
  joined_at  TIMESTAMP  DEFAULT NOW(),
  removed_at  TIMESTAMP,
  PRIMARY KEY (room_id, org_user_id)
);

CREATE TABLE tender_chat_messages (
  message_id  SERIAL    PRIMARY KEY,
  room_id    INT       NOT NULL REFERENCES tender_chat_rooms(room_id) ON DELETE
CASCADE,
  sender_id  INT       NOT NULL REFERENCES organization_employees(org_user_id),
  message    TEXT,
  sent_at    TIMESTAMP  DEFAULT NOW(),
```

```
  is_deleted  BOOLEAN    DEFAULT FALSE
);

CREATE TABLE tender_chat_seen (
  message_id  INT       NOT NULL REFERENCES tender_chat_messages(message_id) ON
DELETE CASCADE,
  org_user_id INT       NOT NULL REFERENCES organization_employees(org_user_id),
  read_at     TIMESTAMP,
  PRIMARY KEY (message_id, org_user_id)
);

-- ============================================================
-- INDEXES
-- ============================================================

-- Users
CREATE INDEX idx_users_email   ON users(email);
CREATE INDEX idx_users_status  ON users(status);

-- Organizations
CREATE INDEX idx_org_type      ON organizations(organization_type);
CREATE INDEX idx_org_verify    ON organizations(verification_status);

-- Tenders
CREATE INDEX idx_tenders_buyer  ON tenders(buyer_id);
CREATE INDEX idx_tenders_status ON tenders(status);
CREATE INDEX idx_tenders_deadline ON tenders(submission_deadline);

-- Bids
CREATE INDEX idx_bids_tender    ON bids(tender_id);
```

```
CREATE INDEX idx_bids_vendor    ON bids(vendor_org_id);
CREATE INDEX idx_bids_status    ON bids(status);

-- Notifications
CREATE INDEX idx_notif_recipients_user ON notification_recipients(org_user_id);
CREATE INDEX idx_notif_recipients_read ON notification_recipients(is_read);

-- Chat
CREATE INDEX idx_tender_chat_messages_room ON tender_chat_messages(room_id);
CREATE INDEX idx_group_chat_org          ON group_chat(org_id);
```

ERD



URL:https://drive.google.com/file/d/1Qt-ku6xim1bqX-XGDbEwuQj6JrKolL4c/view?usp=sharing